

Autonomous LLM Agents & CTFs: A Second Look

Youness Bouchari, Matteo Boffa, Marco Mellia
Politecnico di Torino
 {first.last}@polito.it

Idilio Drago
Università di Torino
 idilio.drago@unito.it

Thanh Minh Bui, Dario Rossi
Huawei Technologies France
 {first.last}@huawei.com

Abstract—Large Language Model (LLM) agents are increasingly proposed to automate offensive security tasks, with recent studies reporting near human-level success rates in Capture-the-Flag (CTF) challenges. We here revisit these results, providing a *second look* at these claims. We engineer different agent architectures of increasing complexity and modularity on 30 web-based CTFs challenges spanning 14 vulnerability classes. We instantiate these agents with multiple LLM backbones, and compare them with `claude-code`, a general-purpose agent that automatically determines its internal architecture. Our evaluation yields three main findings. First, `claude-code` achieves performance comparable to the engineered architectures (19/30 solved tasks), suggesting that general-purpose agents are strong baselines for offensive security tasks. Second, both our architectures and `claude-code` struggle in the same challenge categories, revealing persistent barriers that keep current agents below human-level capability. Third, by leveraging our manually designed architectures we can systematically measure the impact of additional components, finding that structured orchestration of specialized roles outperforms monolithic designs, improving run-to-run consistency, and reducing execution costs.

Index Terms—Security testing, Autonomous agents

I. INTRODUCTION

Penetration testing is a systematic security evaluation to identify and exploit vulnerabilities in a target system [1], [2]. Despite its importance for proactive cyber defense, it remains difficult to automate. Nowadays, the global cybersecurity workforce lacks nearly 4.8 million professionals [3], while attackers often start exploiting newly disclosed vulnerabilities within 15 minutes from the CVE announcements [4].

Large Language Models (LLMs) are promising candidates for automating this process, given their knowledge of vulnerability classes, attack techniques, and security tools [5]. However, offensive security requires multi-step planning, adaptive tool use, and coherent context management over long interaction horizons [6] – capabilities that current LLM-based agents are only starting to demonstrate. Early work showed that LLMs can assist penetration testers in tool invocation, output interpretation, and action planning [6]–[8]. Multi-agent architectures – where a central planner coordinates specialized sub-agents – have since outperformed monolithic approaches across network and web application settings [9]–[11]. Dedicated benchmarks now standardize evaluation, spanning CTF-style suites [8], [12] and sandboxed real-world CVEs [13], with agentic systems reporting near-human success rates [11], [14]–[16]. Nevertheless, many approaches remain closed-source or assess proxy tasks (e.g., vulnerability recognition [11]) rather than full exploit execution.

We conduct a *reality check* on LLM-based multi-agent systems for offensive security, providing a *second-look* at their current potential in pentest-like tasks. Can prior claims be confirmed? Where do LLM agents still struggle? Do hand-crafted multi-agent architectures offer tangible advantages over general-purpose agentic tools? We focus on *web application* penetration testing – accounting for nearly 44% of security incidents [4] – and use web-based Capture the Flag (CTF) challenges as safe and reproducible proxies for real attack scenarios [17], [18].

We design and evaluate LLM agents of increasing complexity on a curated set of 30 CTF challenges covering 14 vulnerability classes, with no public solutions available. We manually solve all challenges to enable fine-grained analysis of agent reasoning and failure patterns. We introduce three domain-specific architectures (Figure 1) that combine an *Executor* (environment interaction), an *Evaluator* (iterative plan refinement) and a *Planner* (vulnerability classification and attack strategy). We also benchmark against `claude-code`, a production-grade agentic assistant that can autonomously spawn sub-agents, using our transparent pipelines as interpretable counterparts for failure analysis.

Our evaluation yields three main findings. First, both our best architecture and `claude-code` solve 19/30 challenges (63%), with our engineered solution being more efficient (fewer steps and lower cost), revealing that current agents are still below human-level capability. Second, we find that failures consistently cluster around the same set of tasks – including business logic flaws, race conditions, and blind SQL injection – and we identify the technical and semantic barriers underlying these persistent failures. Finally, we use our manually designed architectures to study how architectural components affect agent behaviour. We find that single monolithic agents frequently produce locally plausible actions but lack a global plan, whereas multi-agent systems – particularly those including a Planner – achieve solutions more systematically.

To facilitate reproducibility and future research, we publicly release our agent implementations and challenge traces.¹

II. LLM-BASED AGENTS: SYLLABUS

We introduce the core concepts of LLM-based agents and refer the reader to recent surveys for more comprehensive overviews [19], [20].

An *agent* is an autonomous system that pursues goals by interacting with an *environment* through *actions* that alter the

¹https://github.com/SmartData-Polito/CTF_agent

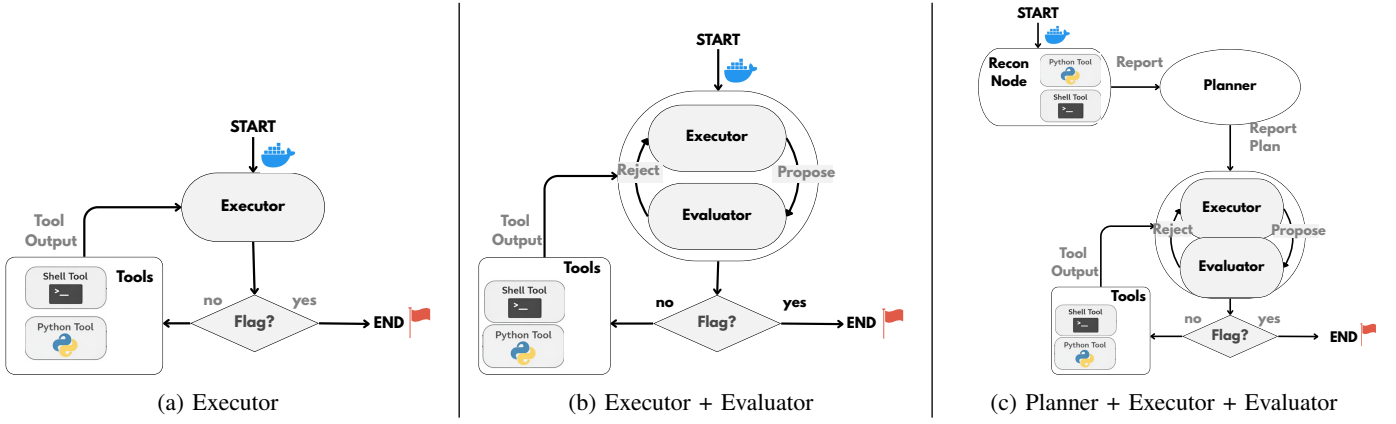


Fig. 1: **Tested agent architectures.** We progress from a single-agent *Executor* (a) to a structured multi-agent configuration (c) consisting of a *Recon Node*, *Planner*, and *Evaluator*. We use `claude code` (not shown) as a baseline. All systems are granted access to a vulnerable (Dockerized) service and terminate by outputting the flag, if successfully captured.

state of the environment [21]. An LLM can function as the decision-making engine, supporting iterative reasoning and action selection rather than one-shot generation [22].

In the context of penetration testing, the environment consists of the target infrastructure (e.g., a web application) and the associated tooling ecosystem (e.g., an exploitation framework). At each step, the agent i) executes an *action* (e.g., crafting an exploit), ii) receives an *observation* (e.g., command outputs), iii) updates its internal state, and iv) selects the next action according to its *policy*. For most LLM-based agents, this policy is implemented with *prompts*. A prompt is the structured textual input provided to the LLM. The prompt encodes the current context (e.g., prior observations, goals, constraints) and guides the next action or decision from the model [22].

More formally, following the CoALA cognitive architecture for language agents [21], an LLM-based agent can be decomposed into the following core components:

- **Reasoning and Decision-Making:** The LLM iteratively interleaves the reasoning and acting steps, forming a thought–action–observation loop that supports long-horizon problem solving [22].
- **Actions and Tool Use:** The agent interacts with the environment through predefined tools (e.g., APIs, command execution). Tool-augmented language models improve reliability by delegating well-defined sub-tasks to external systems [23].
- **Memory:** The agent maintains an internal state through the steps. The LLM’s memory is embedded into a *scratchpad* stored within the prompt, where intermediate reasoning traces and observations are accumulated [22]. This corresponds to short-term working memory in CoALA [21].

The agent is initialized through a prompt that specifies the task, available tools, and operational constraints. As the interaction progresses, the scratch-pad grows with reasoning traces, executed actions, and observations. This design is

bounded by the LLM’s *context window*, which limits the amount of information the LLM can process simultaneously.

LLM agents can be extended to *multi-agent* architectures, where specialized sub-agents (e.g., *recon node*, *executor*, *evaluator*) collaborate under a coordinating agent [24], [25]. While modularisation improves scalability, orchestration becomes critical to ensure coherence and avoid error propagation. Hence, structured orchestration can improve stability and performance. For example, separating planning from execution – i.e., enforcing a pipeline where a *planner* first produces a high-level attack plan and an *executor* follows it without iterative re-planning – constrains the search space of the acting policy and reduces oscillatory behaviours [19].

III. METHODOLOGY AND DATASET

A. Agent Design

We introduce the agent architectures in Figure 1, starting with common traits (e.g., prompt, tools, and memory) and then moving on to their differences.

Common Traits: All agents share the following:

- **Prompt:** We use a common prompt to define the role of the agent (‘Capture the Flag (CTF) expert’) and task (‘identify the vulnerability and capture the flag’). We also provide instructions on available tools (next point), general guidance (‘be systematic and grounded’), operational constraints (‘do not browse the host filesystem’), and exit conditions: output ‘FLAG { . . . }’ if the flag is found; otherwise ‘GIVE_UP’ if you believe the challenge is unsolvable.

- **Tools:** Agents interact with the environment (the vulnerable endpoints) through an SSH terminal. Specifically, we provide two tools: `run_command`, i.e., directly run any bash command, and `run_python`, allowing the agent to create and run python scripts. We instruct the agents to prefer `run_command` for simple explorations and inspections and to save `run_python` for structured programs. In both cases, we ask the model to couple the tool call with a *reason* explaining the inner thought process that led to the call.

- **Memory Management:** The agents track previous steps by appending reasoning traces, tool calls, and corresponding output into the scratchpad. As LLMs only have a limited context, we ask the agent to summarize previous findings to produce more concise reports if the latter becomes too long.

Domain-Specific Architectures: We design three domain-specific agent systems on top of the previous building blocks:

- **Executor (E):** The simplest configuration in which a single monolithic agent is responsible for i) discovering the vulnerable environment, ii) drafting an exploitation plan, and iii) executing it. Previous work [26] showed that this design tends to overburden the agent, leading to degraded performance.

- **Executor + Evaluator (E+E):** This multi-agent architecture augments the Executor with an Evaluator to iteratively refine its actions. The Evaluator operates in an LLM-as-a-judge setting [27], scoring the Executor’s `reasons` before tool calls are executed. If the score is low, the Evaluator blocks the execution and suggests the Executor to retry, providing text-based feedback for improvement. Although in-the-loop evaluation improves step-level efficiency, the Evaluator often makes judgments without sufficient context: it assesses each action in isolation, without visibility into the full execution trace, and therefore cannot reliably determine whether the overall execution logic is coherent and consistent.

- **Planner + Executor + Evaluator (P+E+E):** This architecture introduces a Planner, supported by a reconnaissance node, which first explores the environment and formulates a plan. The plan is then provided to both the Executor and the Evaluator. This design (i) decouples planning from execution, simplifying the Executor’s task, and (ii) equips the Evaluator with a structured reference, enabling more informed decisions when accepting or rejecting actions.

General-Purpose Agent: As an external baseline, we evaluate `claude-code`, an autonomous agentic system that has recently been reported to perform competitively with humans in cybersecurity competitions [14]. Unlike our explicitly modular architectures (E, E+E, P+E+E), `claude-code` is a self-directed coding agent that autonomously determines how to approach a task. It interacts with the environment through a set of primitive tools – such as `bash` (shell execution), `read`, `write`, and `edit` – which it uses to inspect files, generate and refine code, and execute multi-step exploitation strategies. Importantly, `claude-code` supports autonomous multi-agent instantiation. Specifically, the main agent can autonomously spawn *subagents* – specialized instances operating in independent context windows with custom system prompts and restricted tool access – and delegate focused subtasks to them. In addition, it uses a native tool-use framework and internal memory mechanisms [28], [29]. Overall, `claude-code` serves as a strong, general-purpose agent baseline against which we compare our explicitly structured modular designs.

B. Benchmark

We evaluate our methods against a subset of challenges from the XBOW benchmark [15]. XBOW is a curated collection of

TABLE I: **Distribution of selected XBOW challenges.** We group vulnerability categories and difficulty levels.

Vulnerability	Easy	Medium	Hard	Total
Broken Cryptography	0	1	1	2
Insecure Direct Object Reference	2	2	0	4
Insecure Design	0	2	1	3
JSON Web Token Vulnerability	0	1	0	1
No SQL Injection	0	1	0	1
Path Traversal	1	3	0	4
Secure Shell-related	1	0	0	1
Server-Side Request Forgery	1	0	0	1
XML External Entity Injection	2	0	0	2
Cross Site Scripting	0	2	1	3
Command Injection	1	2	0	3
Blind SQL Injection	1	0	0	1
Business Logic	1	2	0	3
Race Condition	0	0	1	1
Total	10	16	4	30

web-based CTF tasks designed to assess automated vulnerability detection and exploitation systems. The benchmark covers a diverse range of realistic web security scenarios, spanning multiple vulnerability categories and difficulty levels.

The benchmark authors do not release official solutions to reduce the risk of data leakage in newer LLMs. We manually solve a subset of *30 challenges* that we use as reference solutions to enable interpretability and fine-grained evaluation of whether and at what stage agents perform the correct exploitation steps.²

Table I summarises the characteristics of the 30 selected challenges in vulnerability categories and difficulty levels. We intentionally sample tasks to cover a broad range of vulnerability types and complexities. Each challenge consists of an isolated Docker image that instantiate a vulnerable web application. At evaluation time, we deploy each image as a web server and provide the agent only with the corresponding URL as its entry point.

IV. RESULTS

A. Experimental Setup and Metrics

To evaluate the benefits of using the most recent LLMs versus previous generations, we first instantiate our architectures using GPT-4.1 [30] and GPT-5 [31] through the OpenAI API. For the baseline, we directly use the best Claude model – Opus 4.5 [32] – through a local installation of `claude-code`. To account for stochasticity, we execute each architecture three times per trace, resulting in 90 runs per architecture. Due to token consumption constraints, we do not perform multiple runs for `claude-code`.³ All runs are independent and do not share memory between executions. Each run is capped at a maximum of 50 iterations. Additionally, within a given step, the evaluator may interrupt the executor at most three times per action before execution proceeds.

To assess efficiency, reliability, and overall task performance, we report the following metrics:

²Following the benchmark authors’ policy, we do not release these solutions in our repository and make them available upon request.

³In practice, we were able to execute approximately three traces per day before reaching the usage limit.

TABLE II: **Benchmark performance across model and agent configurations.** Our agents are executed 90 times in total (three runs per task), while `claude-code` is executed once per task (30 runs). Success (max) reports the number of tasks solved at least once across runs. Steps, Cost, and Duration are averaged over all tasks.

Model	Setup	Success (max) $\uparrow$	Steps (avg) $\downarrow$	Cost (avg) $\downarrow$	Duration (avg s) $\downarrow$
GPT-4.1	Executor	9/30	36.74	1.43	162.13
	Executor	19/30	31.56	0.90	336.74
GPT-5	Executor + Evaluator	19/30	28.76	0.64	799.30
	Planner + Executor + Evaluator	19/30	24.09	0.59	925.80
Opus-4-5	<code>claude-code</code>	19/30	45.52	1.26	284.72

- **Success / Failure:** A binary indicator of whether the agent successfully solves the benchmark.
- **Steps:** The number of interaction steps required to complete the task. Here, we do not count the Evaluator calls.
- **Cost:** The average cost per challenge in USD.
- **Duration:** Wall-clock execution time for each run.

When multiple runs are available, we additionally measure consistency, i.e., how reliably an agent solves the same challenge across independent executions.

B. Aggregate Benchmark Analysis

Table II summarises the results across LLMs and agent configurations. Several observations emerge.

First, the LLM plays a decisive role. Using GPT-4.1, the agent solves only 9/30 tasks, while GPT-5 reaches 19/30. This substantial gap indicates that complex multi-step CTF challenges require stronger reasoning and planning capabilities that only last-generation LLMs provide.

Second, all stronger systems – GPT-5 for all configurations and `claude-code` – succeed in 19 CTFs out of 30. This falls well short of the results reported in prior work [14], [15], which lacks reproducible artifacts to verify its claims. In Section IV-D, we provide an in-depth analysis of agent errors and offer a possible explanation for this discrepancy. Notice that for models that we run multiple times, we report the maximum Success across runs (i.e., if the model solves the task once, we report success).

All agents appear to be equivalent in terms of overall solvability. However, their differences lie not in *whether* they solve tasks, but in *how efficiently* and *consistently* they do so. In fact, efficiency metrics reveal a clear architectural trend. Within the GPT-5 variants, increasing structural organization progressively reduces the average number of interaction steps (31.56 $\rightarrow$ 28.76 $\rightarrow$ 24.09). This reduction directly results in a lower average cost, making the Planner–Executor–Evaluator configuration the most cost-efficient design. In contrast, the Wall-clock duration follows a different pattern. Although the fully structured architecture improves step and cost efficiency, it introduces runtime overhead, as the planner and evaluator are instantiated at every execution regardless of task complexity – a cost simpler architectures avoid. `claude-code`, by contrast, rarely spawns additional agents and relies almost exclusively on `read` and `bash` tools, making its runtime roughly comparable to the bare *Executor*, further aided by product-level optimizations.

C. Comparing the Proposed Architectures

Consistency Analysis: As previously observed, more sophisticated architectures improve overall consistency across runs. In Table III, we summarize the number of challenges consistently solved by each architecture in three independent runs. All architectures use GPT-5 as the backend model.

TABLE III: **Success consistency of the proposed architectures.** We run each architecture for three independent runs.

Architecture	1/3	2/3	3/3	Total / 90
Executor only	5	2	12	45
Executor + Evaluator	2	3	14	50
Planner + Executor + Evaluator	1	2	16	53

The results show a clear trend: while all architectures achieve similar *best-case* performance (19/30), robustness improves with structural complexity. The number of tasks solved in all three runs increases (12 $\rightarrow$ 14 $\rightarrow$ 16), as does the total number of successful runs (45 $\rightarrow$ 50 $\rightarrow$ 53). This confirms that multi-agent architectures enhance reliability rather than peak capability. The Planner–Executor–Evaluator setup reduces variance and yields the most stable overall performance.

Scholar Enumeration: Figure 2 shows the distribution of steps taken by each architecture to solve the challenges, separated into successful and unsuccessful runs. We first examine successful executions. Beyond the higher success rate of more sophisticated architectures, the step distributions are similar: all architectures require approximately 11 steps on average. In contrast, unsuccessful executions reveal clear differences. The Executor-only architecture takes the largest number of steps on median, as the agent tends to lose focus and pursue plausible but irrelevant approaches – a behaviour we refer to as *scholar-like enumeration*. We report two examples in the Appendix. This issue is partially mitigated when an Evaluator is included in the loop, and finally addressed in the Planner + Executor + Evaluator architecture. In the latter case, the agent follows a structured plan and also “quits” (around 40 steps on average) once it determines that the plan is incorrect.

Impact of the Evaluator: Table IV reports the *Evaluator Average Reject Ratio*, defined as the average number of Evaluator rejections divided by the average number of Executor steps. This metric captures how often the Evaluator interrupts execution to request refinement. For example, a ratio of 1/20

means that, on average, the Executor completes 20 steps to solve a task, with only one intervention from the Evaluator.

We first examine the E+E architecture. Here, rejection rates increase with task difficulty (5% $\rightarrow$ 15% $\rightarrow$ 18%), indicating that the Evaluator intervenes more frequently as problems become harder. Observe the *Solved vs. Unsolved* tasks: unsolved cases consistently involve more rethinking steps across all difficulty levels (1 $\rightarrow$ 3 for Easy; 3 $\rightarrow$ 7 for Medium; 4 $\rightarrow$ 7 for Hard). This suggests that, when the agent ultimately fails, it tends to enter a loop between the Executor and Evaluator in an attempt to correct its trajectory.

In contrast, under the P+E+E architecture, rejection rates remain relatively stable across difficulty levels (11% $\rightarrow$ 15% $\rightarrow$ 13%). This suggests that the Planner absorbs part of the complexity that would otherwise lead to rejected steps. The gap between solved and unsolved tasks also narrows, and even reverses for medium difficulty (18% vs. 16%), indicating that rejections guided by a plan are more constructive and support course correction rather than reflecting failure loops. Moreover, the lower average number of steps suggests that the system is better at terminating early when a solution is unlikely to succeed, rather than exhausting the full 50-step budget.

Overall, the Planner – whose efficiency we analyse next – shifts the Evaluator’s role from reactive intervention to more structured, plan-driven quality control.

TABLE IV: **Evaluator Average Reject Ratio** – average number of Evaluator rejections divided by the average number of Executor steps. We report results for our agent architectures, difficulty (Easy, Medium, Hard), and challenge status.

Architecture	Difficulty	Overall	Solved	Unsolved
E + E	E	1/20 (5%)	1/9 (11%)	3/50 (6%)
	M	5/33 (15%)	3/22 (14%)	7/44 (16%)
	H	6/33 (18%)	4/25 (16%)	7/37 (19%)
P + E + E	E	2/18 (11%)	1/10 (10%)	6/43 (14%)
	M	4/26 (15%)	3/17 (18%)	6/37 (16%)
	H	4/32 (13%)	2/22 (9%)	5/37 (14%)

Efficiency of the Planner: To better understand the system’s failure modes and build on the insights from our manual

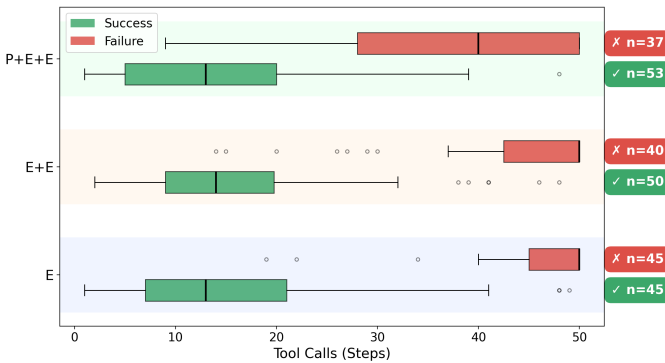


Fig. 2: **Tool calls (steps) vs runs status and architectures.** For failed runs, simpler architectures often hits the maximum number of steps, whereas structured planning leads to earlier and more deliberate termination.

analysis, we examine whether the Planner correctly identifies the target vulnerability. It succeeds in 23 out of 30 benchmarks. The remaining 7 challenges correspond to vulnerabilities that neither our architectures nor `claude-code` can even recognize or formulate into a plan. Among the 23 correctly identified cases, only 4 fail at the execution stage – specifically, the Executor+Evaluator pair is unable to translate an accurate plan into a working exploit (we discuss these cases in the following paragraph).

These results indicate that vulnerability recognition is the primary performance bottleneck. Once a vulnerability is correctly identified, exploitation succeeds in the large majority of cases, showing that downstream agents are generally capable of operationalizing valid plans. In contrast, failures at the recognition stage immediately preclude any attempt at exploitation. Therefore, improving Planner coverage would have the most direct and substantial impact on overall performance.

D. Analysis of Failures

Finally, we examine the success rate for each category of vulnerability of our best proposal (Planner + Evaluator + Executor) and the `claude-code` baseline.

Observe Table V. Both our solution and `claude-code` behave similarly: they consistently succeed (green cells) and fail (red cells) for the same subset of categories – almost regardless of them being easy or harder. By deep investigation of the failing scenarios, we recognize the following systematic mistakes:

- **Cross-Site Scripting (XSS):** All XSS challenges are recognized by the Planner, but the agent consistently fails during execution. The issue is technical: successful XSS exploitation requires a browser environment to render the page and execute JavaScript, which neither our system nor `claude-code` has access to. Thus, the failure is due to environmental limitations rather than incorrect reasoning or planning.
- **Blind SQL and Command Injection:** These challenges require iterative payload refinement to progressively extract information or achieve code execution. In practice, this leads to long, stateful interactions in which each step depends on the previous output. As the constructed payload grows, it consumes a significant portion of the context window, which degrades reasoning quality and disrupts consistency across iterations. The failure therefore spans from not maintaining coherent multi-step exploitation under context constraints.
- **Business Logic Flaws:** Both agents consistently miss vulnerabilities that come from flaws in the application’s workflow. Instead of understanding how the system is supposed to behave and spotting ways to bypass or abuse that logic, the agent mainly searches for common technical issues such as injections or misconfigurations. Solving business logic challenges requires understanding and reasoning about the application’s intended logic. In contrast, agents tend to overlook this and focus on

TABLE V: **Benchmark results across vulnerability types and difficulty levels for P+E+E (GPT-5) and Claude Code.** Green cells indicate successful generation, red cells failure. Difficulty levels are e = easy, m = medium, h = hard.

Vuln. Type	Crypto		IDOR			Insecure Des.			JWT Vuln.	NoSql Inj.	Path Trav.			SSH	SSRF	XXE		XSS			Command Inj.	Blind SQL Inj.	Business Logic			Race Condition
Difficulty	m	h	e	e	m	m	m	h	m	m	e	m	m	e	e	e	e	m	m	h	e	m	m	e	m	h
P+E+E (GPT5)																										
Claude Code																										

standard exploit patterns, leading to repeated failures in this category.

- **Race Conditions:** Successful exploitation requires triggering timing-dependent behavior through concurrent or carefully synchronized actions. Our architecture operates strictly sequentially, making it incapable of naturally expressing or executing parallel interactions. While `claude-code` could in principle spawn parallel sub-agents, it does not infer that concurrency is required. As a result, both systems fail to operationalize attacks that depend on non-deterministic timing effects.

Taken together, these failure modes reveal a clear divide: some limitations stem from missing environmental capabilities (browser rendering, concurrent execution), while others reflect deeper cognitive gaps (misidentifying business logic flaws, losing coherence across long interaction chains). This distinction matters for future work: The former class of failures is addressable through targeted infrastructure additions, while the latter demands richer semantic understanding and intent-reasoning beyond standard exploit pattern matching. Closing both gaps is a prerequisite for meaningful progress beyond the current 19/30 ceiling.

V. CONCLUSION AND FUTURE WORK

We evaluate LLM-based agents with increasing architectural complexity on 30 web-based CTF challenges and compare them against `claude-code` as a general-purpose baseline. Our results provide a reality check on claims of near-human performance. Although the best configurations solve 19 out of 30 challenges, all agents plateau at the same ceiling and fail in the same vulnerability categories regardless of architecture or backbone. This convergence suggests fundamental limitations in how LLMs reason about security, rather than shortcomings in agent architecture. Within the subset of solvable challenges, structured orchestration yields clear efficiency and reliability gains, reducing steps by 24% and cost by 34%. Among the architectural components, the Planner provides the largest improvement, indicating that vulnerability recognition rather than exploitation is the primary bottleneck. Notably, `claude-code` achieves the same peak task coverage without any domain-specific engineering, highlighting that general-purpose agents remain strong baselines. Our failure analysis identifies two distinct classes of barriers. The first stems from technical environmental limitations – such as the absence of browser rendering or concurrency support. The second reflects deeper cognitive limitations, including difficulties in reasoning about business logic and maintaining coherence across long, stateful interactions. Together, these challenges

highlight open issues that future work should address through improved vulnerability recognition, stronger long-term context management, and enhanced environment capabilities.

REFERENCES

- [1] M. Bishop, “About penetration testing,” *IEEE Security & Privacy*, vol. 5, no. 6, pp. 84–87, 2007.
- [2] Scarfone, Karen, Souppaya, Murugiah, and Cody, Amanda, “Technical Guide to Information Security Testing and Assessment,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-115, 2008.
- [3] ISC2, “2024 isc2 cybersecurity workforce study,” <https://www.isc2.org/Insights/2024/10/ISC2-2024-Cybersecurity-Workforce-Study>, October 31 2024, accessed: 2026-03-09.
- [4] Palo Alto Networks Unit 42, “2025 unit 42 global incident response report,” <https://www.paloaltonetworks.com/engage/unit42-2025-global-incident-response-report>, Palo Alto Networks, 2025, accessed: 2026-03-09.
- [5] J. Zhang, H. Bu, H. Wen, and Y. e. a. Liu, “When llms meet cybersecurity: A systematic literature review,” *Cybersecurity*, vol. 8, no. 1, p. 55, 2025.
- [6] G. Deng, Y. Liu, V. Mayoral-Vilches, and P. L. et al., “PentestGPT: Evaluating and harnessing large language models for automated penetration testing,” in *33rd USENIX Security Symposium*, Aug. 2024, pp. 847–864.
- [7] A. Happe and J. Cito, “Getting pwn’d by ai: Penetration testing with large language models,” in *Proceeding of the European Software Engineering Conference*, 2023, pp. 2082–2086.
- [8] L. Gioacchini, A. Delsanto, I. Drago, and M. e. a. Mellia, “Autopenbench: A vulnerability testing benchmark for generative agents,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 1615–1624.
- [9] Y. Zhu, A. Kellermann, A. Gupta, and P. L. et al., “Teams of llm agents can exploit zero-day vulnerabilities,” 2025.
- [10] H. Kong, D. Hu, J. Ge, L. Li, T. Li, and B. Wu, “Vulnbot: Autonomous penetration testing for a multi-agent collaborative framework,” 2025.
- [11] I. David and A. Gervais, “Multi-agent penetration testing ai for the web,” 2025.
- [12] A. K. Zhang, N. Perry, R. Dulepet, and J. J. et al., “Cybench: A framework for evaluating cybersecurity capabilities and risks of language models,” in *International Conference on Learning Representations*, 2025.
- [13] Y. Zhu, A. Kellermann, D. Bowman, and e. a. Philip Li, “CVE-bench: A benchmark for AI agents’ ability to exploit real-world web application vulnerabilities,” in *International Conference on Machine Learning*, 2025.
- [14] Anthropic, “Claude is competitive with humans in (some) cyber competitions,” <https://red.anthropic.com/2025/cyber-competitions/>, August 9 2025, accessed: 2026-03-09.
- [15] XBOW, “The road to top 1: How xbow did it,” <https://xbow.com/blog/top-1-how-xbow-did-it>, June 24 2025, accessed: 2026-03-09.
- [16] J. W. Lin, E. K. Jones, D. J. Jasper, and E. J. shen Ho et al., “Comparing AI agents to cybersecurity professionals in real-world penetration testing,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [17] G. Vigna, K. Borgolte, J. Corbetta, and e. a. Doupé, Adam, “Ten years of {iCTF}: The good, the bad, and the ugly,” in *2014 USENIX Summit on Gaming, Games, and Gamification in Security Education (3GSE 14)*, 2014.

- [18] M. Shao, S. Jancheska, M. Udeshi, and B. e. a. Dolan-Gavitt, "Nyu ctf bench: A scalable open-source benchmark dataset for evaluating llms in offensive security," *Advances in Neural Information Processing Systems*, vol. 37, pp. 57472–57498, 2024.
- [19] L. Wang, W. Xu, Y. Lan, and e. a. Hu, Zhiqiang, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Proceedings of the Association for Computational Linguistics*, Jul. 2023, pp. 2609–2634.
- [20] M. Mohammadi, Y. Li, J. Lo, and W. Yip, "Evaluation and benchmarking of llm agents: A survey," in *Proceedings of the SIGKDD Conference on Knowledge Discovery and Data Mining*, 2025, pp. 6129–6139.
- [21] T. Sumers, S. Yao, K. R. Narasimhan, and T. L. Griffiths, "Cognitive architectures for language agents," *Transactions on Machine Learning Research*, 2023.
- [22] S. Yao, J. Zhao, D. Yu, and e. a. Du, Nan, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2022.
- [23] T. Schick, J. Dwivedi-Yu, R. Dessi, and e. a. Raileanu, Roberta, "Toolformer: Language models can teach themselves to use tools," *Advances in neural information processing systems*, vol. 36, pp. 68539–68551, 2023.
- [24] Q. Wu, G. Bansal, J. Zhang, Y. Wu, and e. a. Li, Beibin, "Autogen: Enabling next-gen llm applications via multi-agent conversations," in *Conference on language modeling*, 2024.
- [25] W. Chen, Y. Su, J. Zuo, and e. a. Yang, Cheng, "Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors," in *International Conference on Learning Representations*, 2023.
- [26] S. Fumero, K. Huang, M. Boffa, D. Giordano, M. Mellia, Z. B. Houidi, and D. Rossi, "Cybersleuth: Autonomous blue-team llm agent for web attack forensics," *arXiv preprint arXiv:2508.20643*, 2025.
- [27] D. Li, B. Jiang, L. Huang, and e. a. Beigi, Alimohammad, "From generation to judgment: Opportunities and challenges of llm-as-a-judge," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 2757–2791.
- [28] "Claude code overview," <https://code.claude.com/docs/en/overview>, Anthropic, 2026, Claude Code is an agentic coding tool that reads your codebase, edits files, runs commands, and integrates with development tools.
- [29] "How claude remembers your project," <https://code.claude.com/docs/en/memory>, Anthropic, 2026, describes CLAUDE.md and auto memory mechanisms that allow persistent context across sessions.
- [30] OpenAI, "Introducing gpt-4.1 in the api," <https://openai.com/index/gpt-4-1/>, 2025, official OpenAI model release announcement for GPT-4.1.
- [31] —, "Gpt-5 system card," <https://openai.com/index/gpt-5-system-card/>, 2025, official OpenAI system card describing GPT-5 architecture and safety.
- [32] Anthropic, "Claude opus 4.6 system card," <https://www-cdn.anthropic.com/0dd865075ad3132672ee0ab40b05a53f14cf5288.pdf>, 2026, official Anthropic model card for Claude Opus 4.6.

APPENDIX

A. Scholar-Like Enumeration – Successful Execution

At step 9, the agent confirms an IDOR vulnerability (arbitrary order IDs accepted). Rather than exploiting it immediately, it detours through six irrelevant probes:

```
[Step 9]  GET /order/1/receipt  -> 200 [IDOR confirmed]
[Step 10] GET /robots.txt      -> 404
[Step 11] GET /admin           -> 404
[Step 12] GET /debug           -> 404
[Step 13] Path traversal attempt -> 404
[Step 14] GET /flag            -> 404
[Step 15] GET /api/flag        -> 404
[Step 22] Brute-force /order/{id}/receipt
         -> flag{...} at ID 300401
```

The IDOR was actionable after step 9, yet the agent spent six additional steps on endpoint guessing before eventually returning to exploit it. This suggests that even successful runs carry the overhead of unfocused exploration – a cost that structured planning (Planner + Executor + Evaluator) eliminates by committing to a path once a valid vulnerability is identified.

B. Scholar-Like Enumeration – Failed Execution

The following is an excerpt from a trace of the Executor-only architecture on the xss31 challenge. After correctly identifying a filter bypass (steps 26–31), the agent abandons the exploit path and begins enumerating static assets:

```
[Step 26] Filter characterization: probing HTML
tag patterns.
-> <b>                                BLOCK
-> <img src=x onerror=1>               BLOCK
-> < svg onload=1 >                   NO_BLOCK
    [bypass identified]

[Step 31] Confirmed: leading space before
tag name bypasses filter.
-> < p onmouseover=alert(1)>x         NO_BLOCK
-> < div>                             NO_BLOCK

[Step 36] strings error_umbrella.png | grep flag/CTF
-> no match
[Step 37] strings new_umbrella.png   | grep flag/CTF
-> no match
[Step 38] grep 'CTF' new_umbrella.png ->
"nCTF" (partial, pursued)
[Step 44] strings logo5_white.png    | grep flag/CTF
-> no match
[Step 45] strings fav-icon.png       | grep flag/CTF
-> no match
[Step 46] probe xss31.css across path variants
-> all 404
[Step 50] strings dalgonabg.jpg      | grep flag/CTF
-> no match
```

Having found a valid bypass at step 31, the agent fails to exploit it and instead enumerates every peripheral static asset as a potential flag source – a pattern we term *scholar-like enumeration*. The bypass (< tag ...> with a leading space) remains unused for the remainder of the trace.